

Intention Estimation + Multimodal Visual Understand + SLAM Navigation + Visual Functions (Text Version)

Intention Estimation + Multimodal Visual Understand + SLAM Navigation + Visual Functions (Text Version)

1. Course Content
2. Preparation
 - 2.1 Content Description
3. Document Configuration
 - 3.1 Creating a xlsx Document
 - 3.2 Uploading to the RAG Knowledge Base
4. Run the Example
 - 4.1 Starting the Program
 - 4.1.1 Jetson Nano Board Startup Steps:
 - 4.1.2 RDKX5, Raspberry Pi PI5 and ORIN board startup steps:
 - 4.2 Test Cases
 - 4.2.1 Example 1
5. Source Code Analysis
 - 5.1 Example 1

1. Course Content

Note: This section requires you to first complete the map file configuration as described in the [Multimodal Visual Understand + SLAM Navigation] section.

1. Learning to Use the RAG Knowledge Base to Train Personal Intention Understanding

Course Review:

The RAG knowledge base can be used to expand the knowledge base of the large model. This section explains how to expand the RAG knowledge base so that the large model can understand personal intentions.

Important Note! !:

1. The expanded personal intention understanding function may vary from user to user, and the large model's responses are divergent, so the actual debugging results may not be exactly the same.
2. The expanded personal intention understanding function must comply with social norms and laws and regulations. Technical Support assumes no responsibility for the final debugging results or any impact of this course.

2. Preparation

2.1 Content Description

This course uses a Raspberry Pi 5 as an example. For Raspberry Pi and Jetson Nano boards, open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this lesson in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and**

Operation Guide -- [5.Enter Docker (For JETSON Nano and RPi 5)]. For Orin boards, simply open a terminal and enter the commands mentioned in this lesson.

This example uses `model:"qwen/qwen2.5-v1-72b-instruct:free","qwen-v1-latest"`

⚠ For the same test command, the model's responses may not be exactly the same and may differ slightly from the screenshots in the tutorial. To increase or decrease the diversity of the model's responses, refer to the section on configuring the decision-making model parameters in [04.AI Large Model Development] -- [5.Deploy the RAG knowledge base].

3. Document Configuration

3.1 Creating a xlsx Document

Open the [Intent Mapping] folder in this lesson's folder and select the corresponding [Intent Mapping_xx.xlsx] file based on your robot configuration.

	A	B	
1	query	answer	
2	I'm in placel right now, and I'm a little thirsty.	1. Navigate to tea room, 2. Check for the presence of bottled water, 3. Navigate to the user's location (placel), 4. Report the observed results back to the user.	
3	I want to sleep	Tell the user a bedtime story to soothe their emotions.	
4	I'm a little hungry.	1. Navigate to kitchen. 2. Observe if there is bread present. 3. If bread is observed, the servo motor nods; otherwise, the servo motor shakes its	
5			

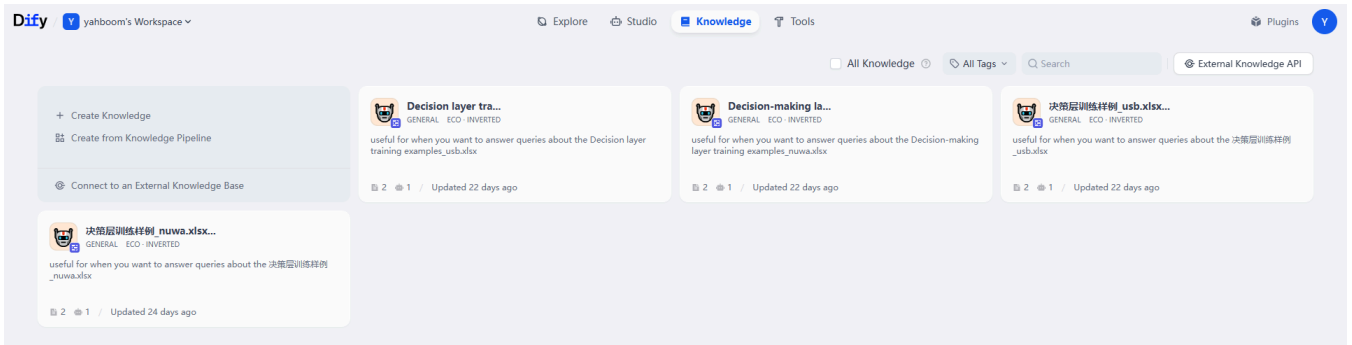
(You can add multiple files and more customized information as needed.)

[Intent Mapping]: Stores personal intentions and expectations for the large language model to control the robot's actions.

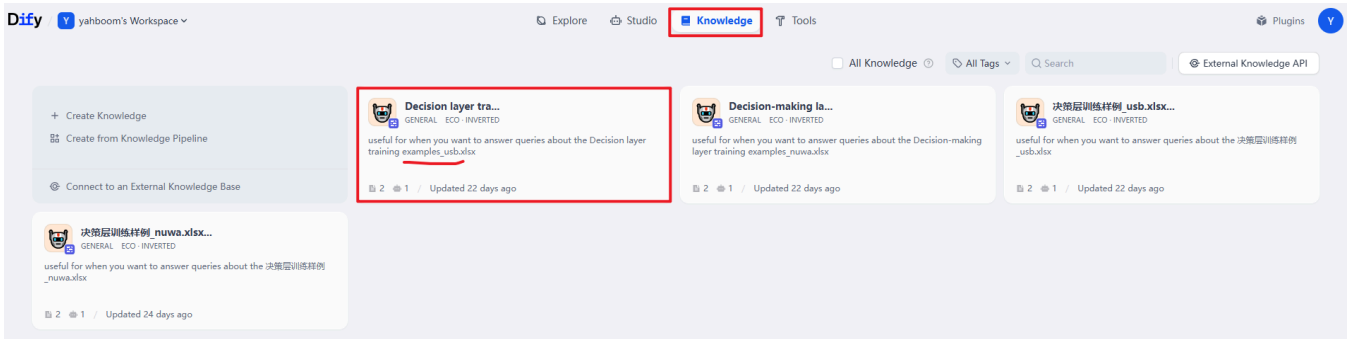
3.2 Uploading to the RAG Knowledge Base

Please modify according to the instructions in the [04.AI Large Model Development] -- [5.Deploy the RAG knowledge base].

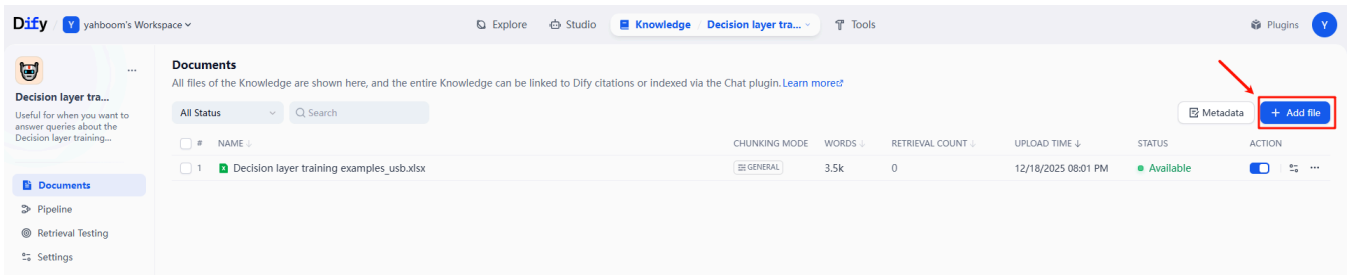
1. Access the IP address of the in-car system to enter the Dify backend.



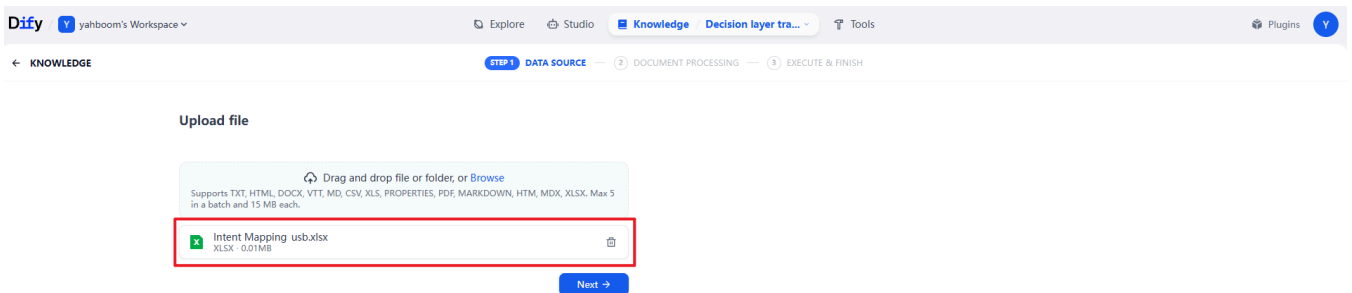
2. Click on "Knowledge Base" and open the knowledge base for the corresponding camera.



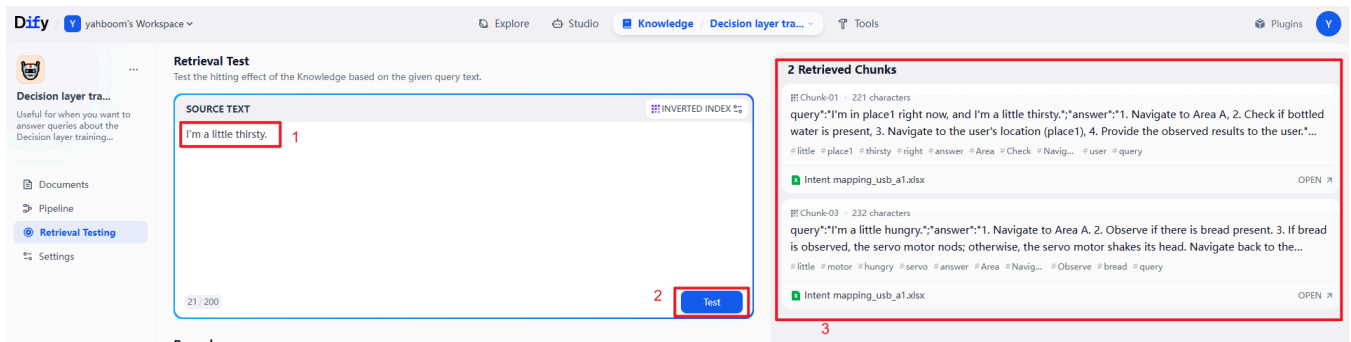
3. Click on "Add File".



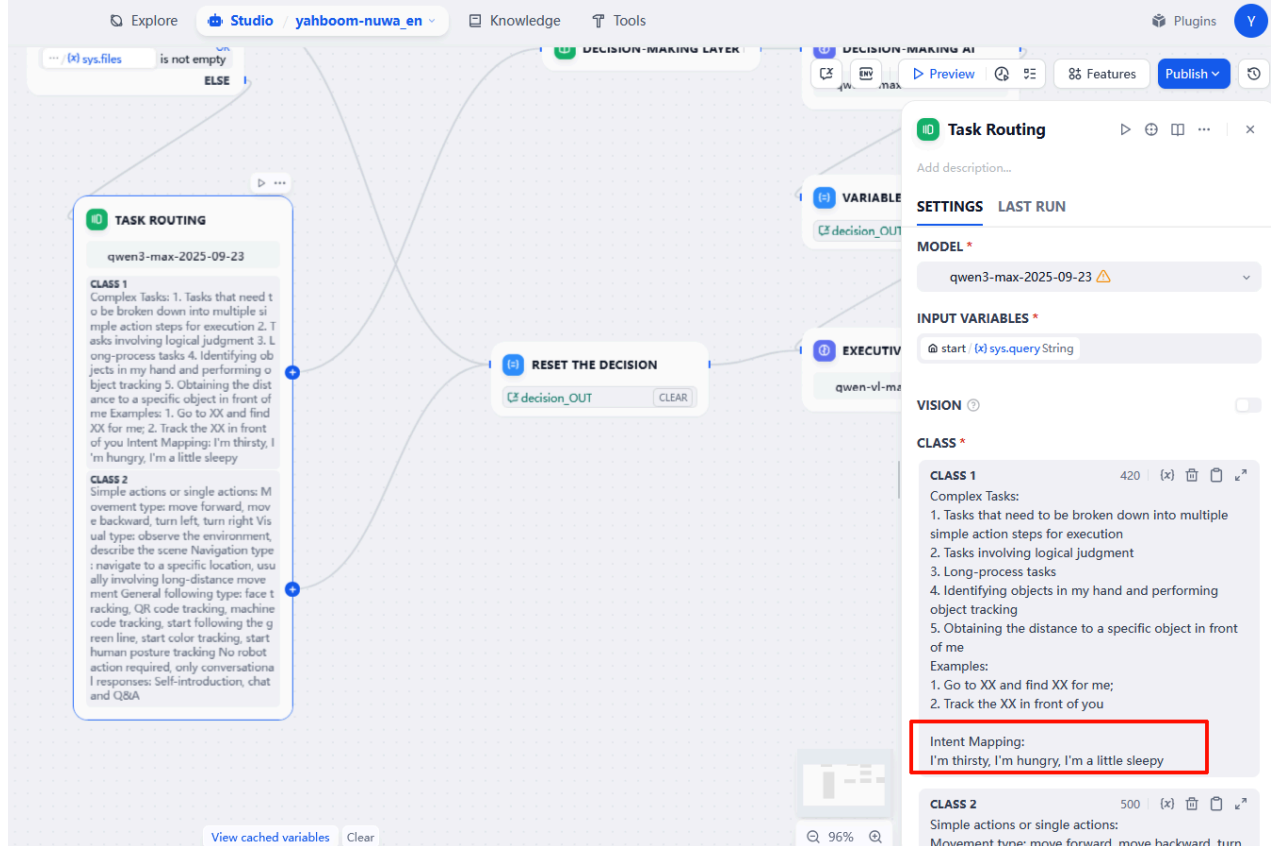
4. Add the prepared "Intent Mapping_xx.xlsx" file.



5. Click "Next", then "Save and Process".

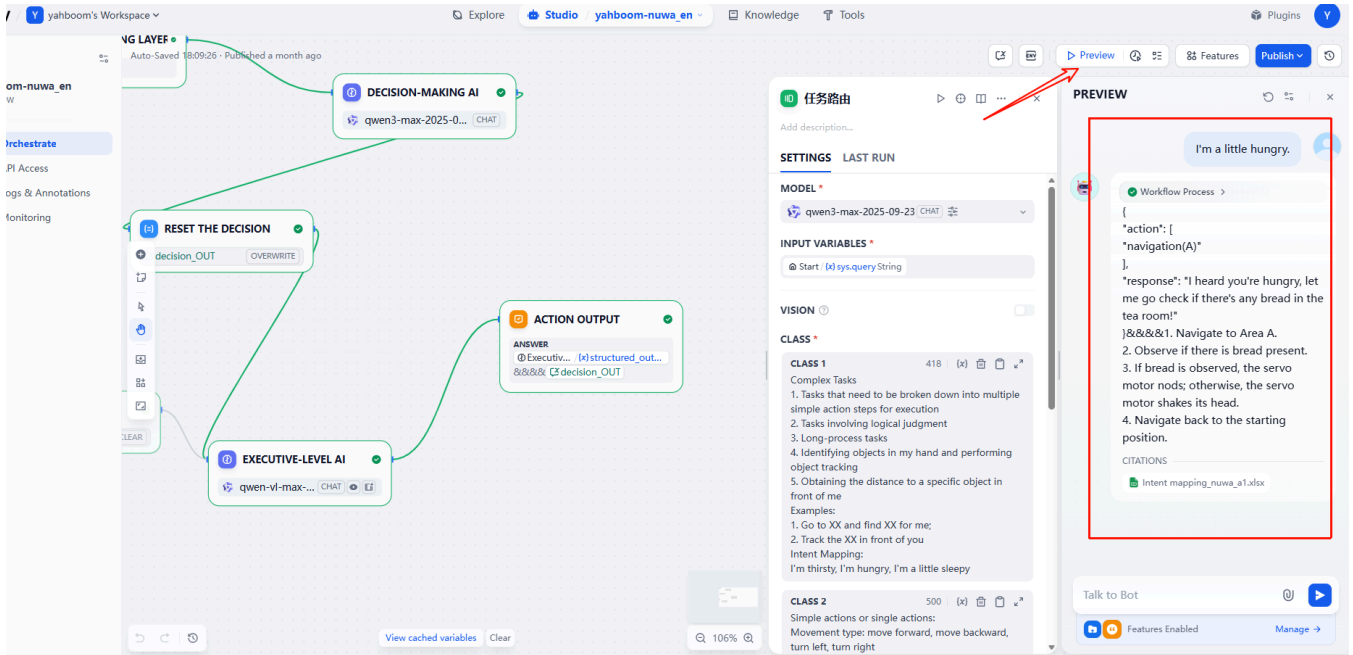


8. Finally, go to the [Studio] interface, select the application corresponding to your camera model (here, we use [yahboom-nuwa_en] as an example).



Select [Task Routing] and add your custom intent mapping here, for example, "[I'm a little hungry]".

Click [Preview] to test it.



Finally, click "Publish" in the upper right corner to republish the corresponding application.

4. Run the Example

4.1 Starting the Program

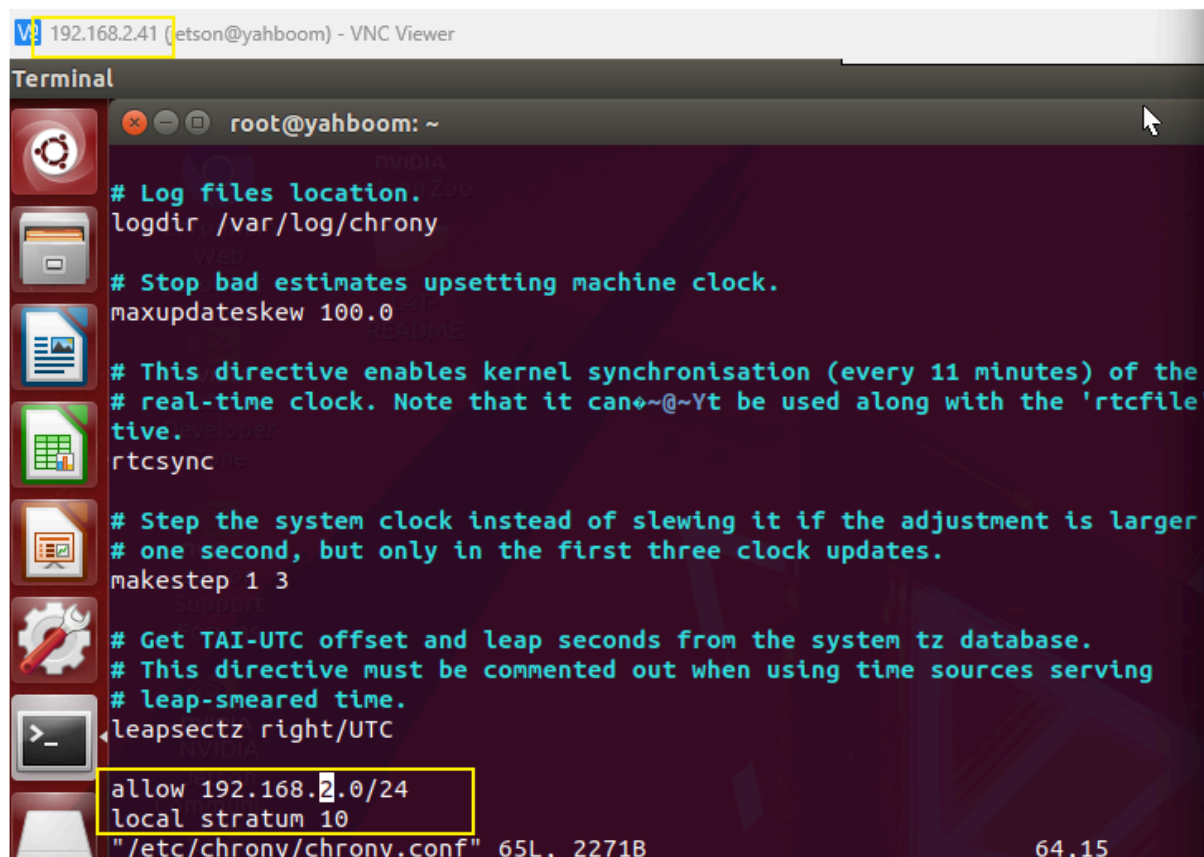
4.1.1 Jetson Nano Board Startup Steps:

Due to Nano performance issues, navigation-related nodes must be run on a virtual machine. Therefore, navigation performance is highly dependent on the network. We recommend running in an indoor environment with a good network connection. You must first configure the following:

- **Board Side (Requires Docker)**

Open a terminal and enter

```
sudo vi /etc/chrony/chrony.conf
```



```
192.168.2.41 (jetson@yahboom) - VNC Viewer
Terminal
root@yahboom: ~
# Log files location.
logdir /var/log/chrony

# Stop bad estimates upsetting machine clock.
maxupdateskew 100.0

# This directive enables kernel synchronisation (every 11 minutes) of the
# real-time clock. Note that it can't be used along with the 'rtcfile'
# directive.
rtcsync

# Step the system clock instead of slewing it if the adjustment is larger
# one second, but only in the first three clock updates.
makestep 1 3

# Get TAI-UTC offset and leap seconds from the system tz database.
# This directive must be commented out when using time sources serving
# leap-smeared time.
leapsectz right/UTC

allow 192.168.2.0/24
local stratum 10

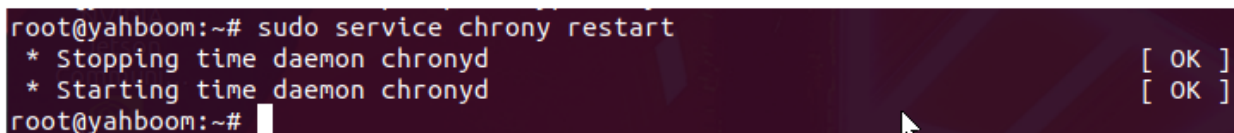
"/etc/chrony/chrony.conf" 65L, 2271B 64,15
```

Add the following two lines to the end of the file: (Fill in 192.168.2.0/24 based on the actual network segment; this example uses the current board IP address of 192.168.2.41.)

```
allow 192.168.x.0/24 #x indicates the corresponding network segment
local stratum 10
```

After saving and exiting, enter the following command to take effect:

```
sudo service chrony restart
```



```
root@yahboom:~# sudo service chrony restart
* Stopping time daemon chronyd [ OK ]
* Starting time daemon chronyd [ OK ]
root@yahboom:~#
```

- **Virtual Machine**

Open a terminal and enter

```
echo "server 192.168.2.41 iburst" | sudo tee /etc/chrony/sources.d/jetson.sources
```

The 192.168.2.41 entry above is the board's IP address.

Enter the following command to take effect.

```
sudo chronyc reload sources
```

Enter the following command again to check the latency. If the IP address appears, it's normal.

```
chronyc sources -v
```

```
yahboom@VM:~$ chronyc sources -v

.-- Source mode '^' = server, '=' = peer, '#' = local clock.
/ .- Source state '*' = current best, '+' = combined, '-' = not combined,
| /      'x' = may be in error, '~' = too variable, '?' = unusable.
||
||      Reachability register (octal) --.      .- xxxx [ yyyy ] +/- zzzz
||      Log2(Polling interval) --.      |      xxxx = adjusted offset,
||                                  |      yyyy = measured offset,
||                                  |      zzzz = estimated error.
||                                  \      /
||                                  |      |
MS Name/IP address             Stratum Poll Reach LastRx Last sample
=====
^- prod-ntp-5.ntp1.ps5.cano>    2  7  277   133  -4048us[-5395us] +/- 132ms
^- alphyn.canonical.com        2  8  177    66   -193us[-193us] +/- 134ms
^- prod-ntp-3.ntp1.ps5.cano>    2  8  367   131   -16ms[-18ms] +/- 127ms
^- prod-ntp-4.ntp4.ps5.cano>    2  8  377   129 -4907us[-6254us] +/- 133ms
^* time.nju.edu.cn             1  7  377    69   +77us[-1270us] +/- 17ms
^+ 139.199.215.251              2  8  377   135 +2358us[+1011us] +/- 46ms
^+ 119.28.183.184               2  7  377   193 +2061us[+713us] +/- 28ms
^+ 119.28.206.193               2  8  367    8  +2058us[+2058us] +/- 36ms
^+ 192.168.2.41                 4  6  377    6   -12ms[-12ms] +/- 92ms
yahboom@VM:~$
```

- Start the program

Open the host machine terminal and start the Dify environment:

```
sh ~/bringup_dify.sh
```

Open the terminal in Docker and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization, the following content will be displayed:

```
[usb_cam_node_exe-1] YUYV 4:2:2 800 x 600 (20 Hz)
[usb_cam_node_exe-1] YUYV 4:2:2 1280 x 1024 (6 Hz)
[usb_cam_node_exe-1] YUYV 4:2:2 320 x 240 (30 Hz)
[usb_cam_node_exe-1] [INFO] [1765965894.552487562] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[usb_cam_node_exe-1] [INFO] [1765965894.552563191] [usb_cam]: Setting 'exposure_auto' to 1
[usb_cam_node_exe-1] [INFO] [1765965894.552578635] [usb_cam]: Setting 'exposure' to 150
[usb_cam_node_exe-1] [INFO] [1765965894.585467996] [usb_cam]: Setting 'focus_auto' to 0
[usb_cam_node_exe-1] [INFO] [1765965894.673818636] [usb_cam]: Timer triggering every 8 ms
[INFO] [model_service-12]: process started with pid [439]
[INFO] [action_service_usb-13]: process started with pid [441]
[action_service_usb-13] [INFO] [1765965908.031574888] [action_service_usb]: ROS_Action_Service Initialization completed
[model_service-12] [INFO] [1765965975.153285944] [model_service]: Dify LLM connection successful
[model_service-12] [INFO] [1765965975.154461154] [model_service]: ROS_LLM_Service Initialization completed
```

On the virtual machine, create a new terminal and start.

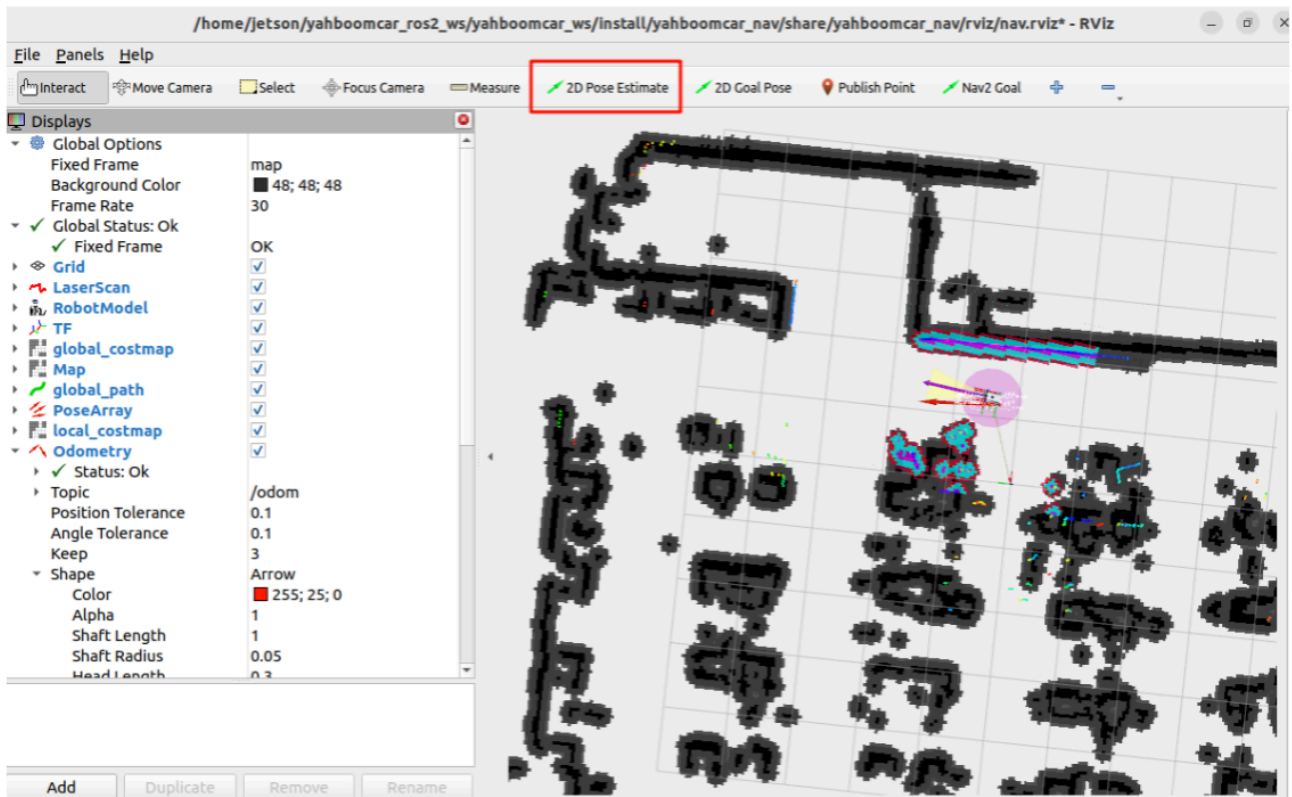
```
ros2 launch yahboomcar_nav display_nav_launch.py
```


Wait for the navigation algorithm to start before displaying the map. Then open a VM terminal and enter:

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
```

Note: When running the car's mapping function, you must enter the save map command on the VM to save the navigation map.

After that, follow the navigation function startup process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to select it. Roughly mark the robot's position and orientation on the map. After initializing positioning, preparations are complete.



4.1.2 RDKX5, Raspberry Pi PI5 and ORIN board startup steps:

Open the host terminal and start the Dify environment.

```
sh ~/bringup_dify.sh
```

```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~ $ sh ~/bringup_dify.sh  
[+] Running 13/13  
✓ Network docker_ssrf_proxy_network Created 0.1s  
✓ Network docker_default Created 0.1s  
✓ Container docker-weaviate-1 Started 2.2s  
✓ Container docker-redis-1 Started 2.6s  
✓ Container docker-web-1 Started 3.1s  
✓ Container docker-sandbox-1 Started 2.2s  
✓ Container docker-db-1 Healthy 4.6s  
✓ Container docker-ssrf_proxy-1 Started 3.1s  
✓ Container docker-worker_beat-1 Started 7.3s  
✓ Container docker-worker-1 Started 7.2s  
✓ Container docker-plugin_daemon-1 Started 6.2s  
✓ Container docker-api-1 Started 6.8s  
✓ Container docker-nginx-1 Started 7.7s  
pi@raspberrypi:~ $
```

For Raspberry Pi 5, you need to first enter the Docker container. For RDKX5 and Orin board, this is not necessary.

Open a terminal in Docker and enter the command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization is complete, the following will be displayed:

```
[usb_cam_node_exe-1] YUYV 4:2:2 800 x 600 (20 Hz)  
[usb_cam_node_exe-1] YUYV 4:2:2 1280 x 1024 (6 Hz)  
[usb_cam_node_exe-1] YUYV 4:2:2 320 x 240 (30 Hz)  
[usb_cam_node_exe-1] [INFO] [1765965894.552487562] [usb_cam]: Setting 'white_balance_temperature_auto' to 1  
[usb_cam_node_exe-1] [INFO] [1765965894.552563191] [usb_cam]: Setting 'exposure_auto' to 1  
[usb_cam_node_exe-1] [INFO] [1765965894.552578635] [usb_cam]: Setting 'exposure' to 150  
[usb_cam_node_exe-1] [INFO] [1765965894.585467996] [usb_cam]: Setting 'focus_auto' to 0  
[usb_cam_node_exe-1] [INFO] [1765965894.673818636] [usb_cam]: Timer triggering every 8 ms  
[INFO] [model_service-12]: process started with pid [439]  
[INFO] [action_service_usb-13]: process started with pid [441]  
[action_service_usb-13] [INFO] [1765965908.031574888] [action_service_usb]: ROS_Action_Service Initialization completed  
[model_service-12] [INFO] [1765965975.153285944] [model_service]: Dify LLM connection successful  
[model_service-12] [INFO] [1765965975.154461154] [model_service]: ROS_LLM_Service Initialization completed
```

Create a new terminal on the virtual machine and start it. (For RDKX5, Raspberry Pi and Jetson Nano, it is recommended to run visualization in the virtual machine)

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Wait for the navigation algorithm to start and the image will be displayed. Then open the Docker terminal and enter

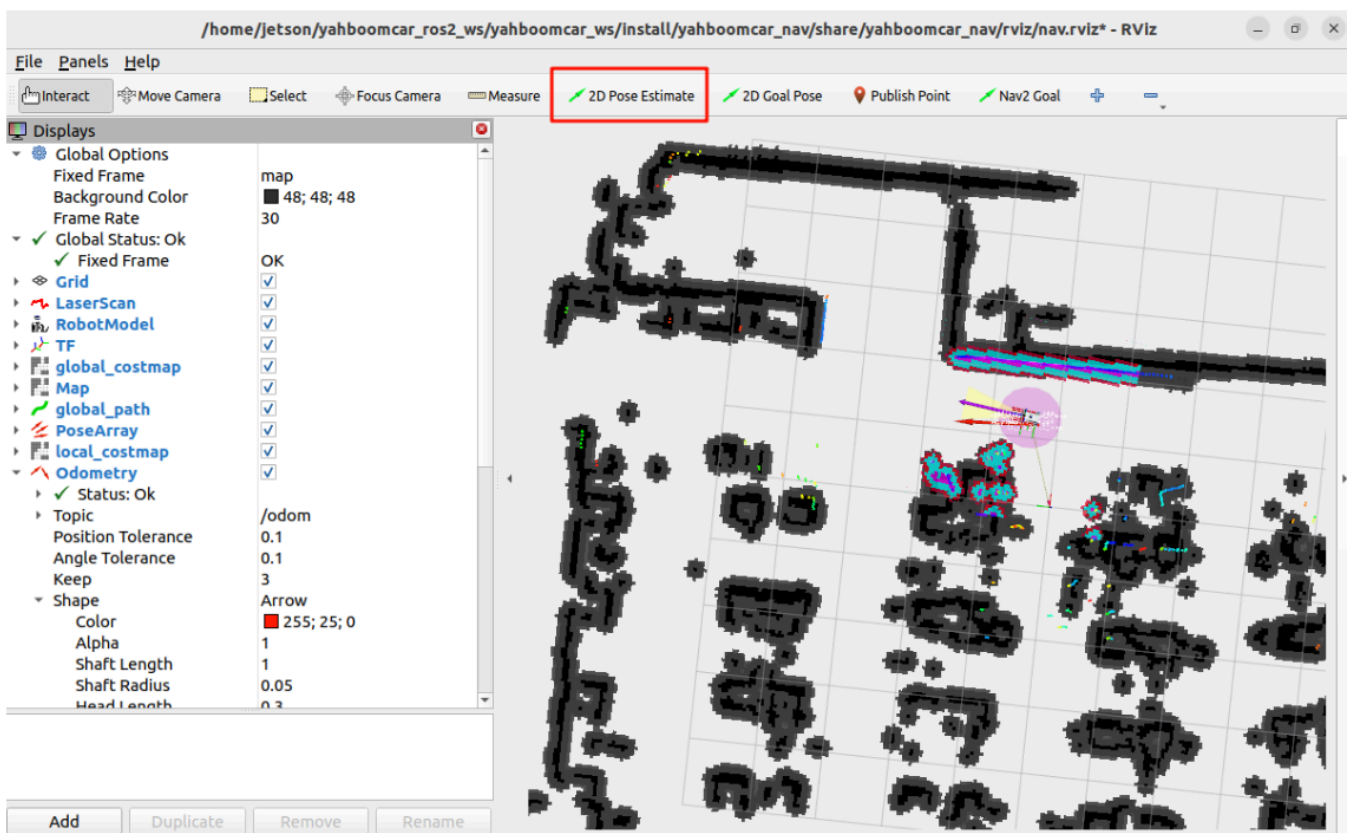
```
#Choose one of the two navigation algorithms
#Normal navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

#Fast Relocation Navigation (Not supported on RDKX5, Raspberry Pi 5, and Jetson Nano main controllers)
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=true
load_state_filename:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.pbstream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.yaml
params_file:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/cartodb_nav_params.yaml
```

Note: yahboomcar.yaml and yahboomcar.pbstream must be mapped simultaneously, meaning they are the same map. Refer to the cartograph mapping algorithm for saving maps.

After that, follow the navigation function startup process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to enter the selection state. Roughly mark the robot's position and orientation on the map. After initializing positioning, preparations are complete.



Log into the same docker run from multiple terminals and start it.

```
ros2 run text_chat text_chat
```

```
root@raspberrypi: /
File Edit Tabs Help
*root@raspb... x *root@raspb... x root@raspb... x root@raspb... x
root@raspberrypi:~# ros2 run text_chat text_chat
user input: I
```

4.2 Test Cases

Here are some test cases for reference; users can create their own dialogue commands.

- I'm in the office and feeling thirsty.

4.2.1 Example 1

Type "I'm in the office now, feeling a little thirsty" in the terminal. The terminal prints the following information:

```
root@raspberrypi: ~
File Edit Tabs Help
root@raspb... x root@raspb... x root@raspb... x pi@raspberr... x
root@raspberrypi:~# ros2 run text_chat text_chat
user input: 我现在办公区，我有点口渴了
okay☺, let me think for a moment... -[INFO] [1755692529.223522879] [text_chat_node]: 决策层AI规划:1. 导航前往厨房区域
2. 观察是否存在矿泉水
3. 如果观察到矿泉水，反馈矿泉水和你之间的距离；否则前进0.5米后退0.5米
4. 导航返回办公区
user input: [INFO] [1755692530.957639015] [text_chat_node]: "action": ['navigation(G)'], "response": 好的，我就去厨房给你找矿泉水，就像一个小助手一样
[INFO] [1755692557.989323529] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达厨房了，现在用我的火眼金睛寻找矿泉水
[INFO] [1755692560.396245271] [text_chat_node]: "action": ['get_dist(240,210)'], "response": 我看到前面有一个矿泉水瓶，正在测量它和我的距离
[INFO] [1755692564.370085347] [text_chat_node]: "action": ['navigation(F)'], "response": 我已经测到矿泉水离我1.004米，现在准备返回办公区啦
[INFO] [1755692590.463255325] [text_chat_node]: "action": ['finishtask()'], "response": 我已经完成全部任务啦，矿泉水在1.004米处，现在安全返回办公区了
I
```

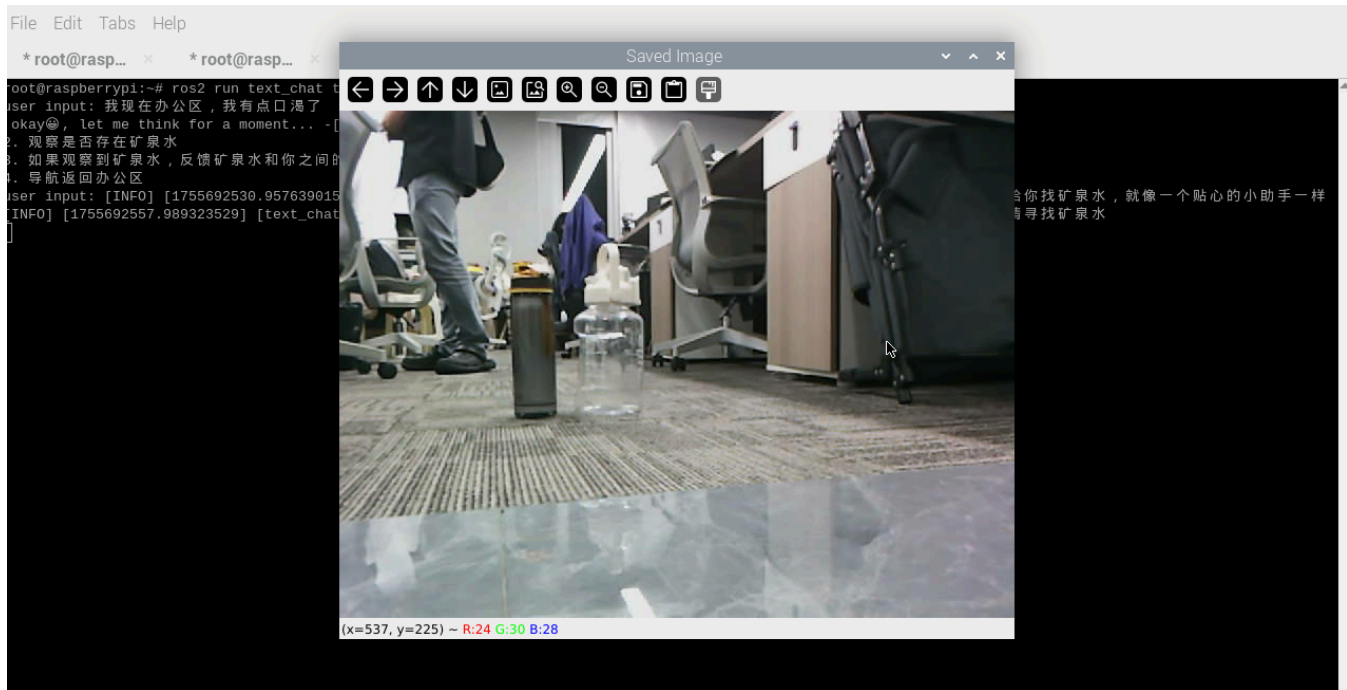
The decision-making model outputs the planned task steps:

```
user input: 我现在办公区，我有点口渴了
okay☺, let me think for a moment... -[INFO] [1755692529.223522879] [text_chat_node]: 决策层AI规划:1. 导航前往厨房区域
2. 观察是否存在矿泉水
3. 如果观察到矿泉水，反馈矿泉水和你之间的距离；否则前进0.5米后退0.5米
4. 导航返回办公区
```

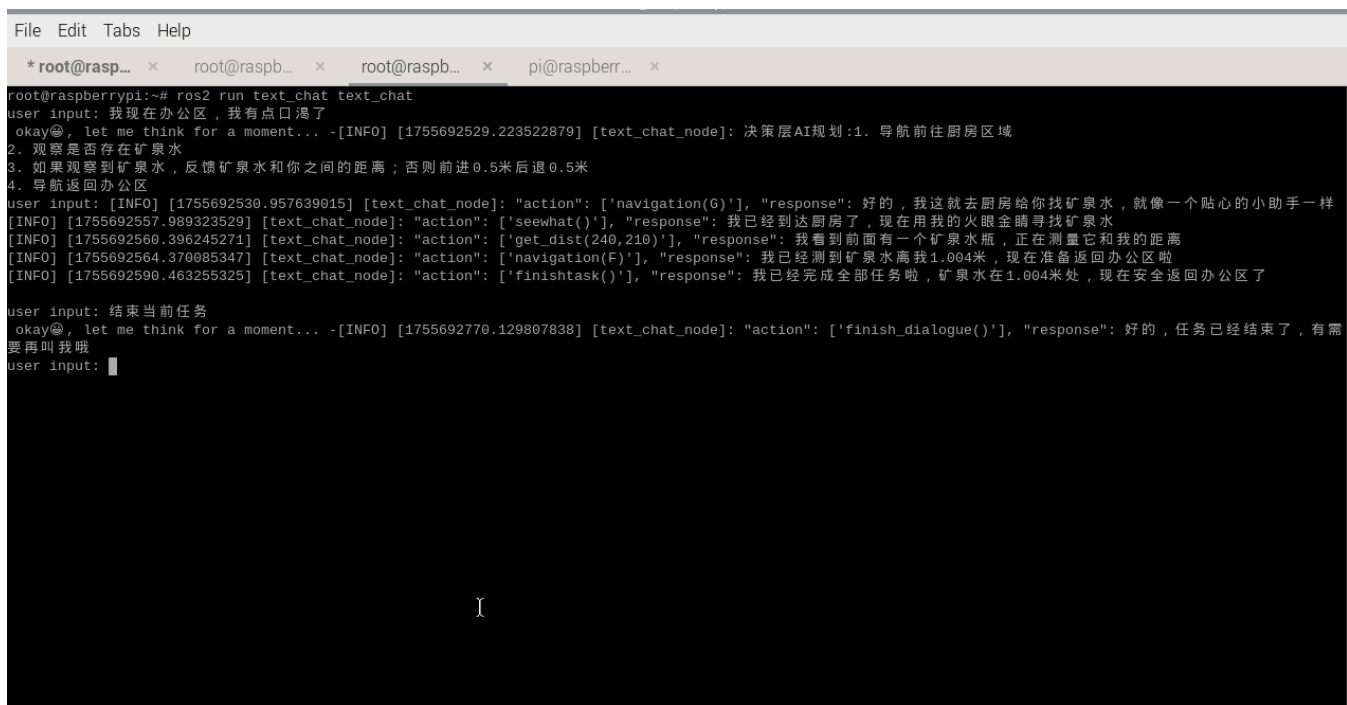
The execution model then executes the task steps:

```
user input: [INFO] [1755692530.957639015] [text_chat_node]: "action": ['navigation(G)'], "response": 好的，我这就去厨房给你找矿泉水，就像一个小助手一样
[INFO] [1755692557.989323529] [text_chat_node]: "action": ['seewhat()'], "response": 我已经到达厨房了，现在用我的火眼金睛寻找矿泉水
[INFO] [1755692560.396245271] [text_chat_node]: "action": ['get_dist(240,210)'], "response": 我看到前面有一个矿泉水瓶，正在测量它和我的距离
[INFO] [1755692564.370085347] [text_chat_node]: "action": ['navigation(F)'], "response": 我已经测到矿泉水离我1.004米，现在准备返回办公区啦
[INFO] [1755692590.463255325] [text_chat_node]: "action": ['finishtask()'], "response": 我已经完成全部任务啦，矿泉水在1.004米处，现在安全返回办公区了
```

A four-second screen appears after arriving at the kitchen.



After completing a task, the robot enters a waiting state. Instructions are passed directly to the execution layer model, and all conversation history is retained. You can press the **ENTER** key in the terminal and continue the conversation by entering the "End current task" command to terminate the current task cycle and start a new one.



5. Source Code Analysis

Source code is located at:

Jetson Orin Nano:

```
#NUWA Camera User
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_nuwa.py
#USB Camera User
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_usb.py
```

Jetson Nano, Raspberry Pi:

You need to first enter Docker

```
#NUWA Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_nuwa.py
#USB Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/largemodel/largemodel/action_service_usb.py
```

RDkX5:

```
#NUWA Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

5.1 Example 1

action_service.py :

This example uses the **seewhat**, **navigation**, **load_target_points**, and **get_dist** methods in the **CustomActionServer** class. **seewhat**, **navigation**, **load_target_points**, and **get_dist** are described in the previous section [2.Multimodal visual understand], [6.Multimodal visual understand + Depth Camera Distance Question Answering], [7.Multimodal visual understand+SLAM navigation] have all been explained, and there are no new procedures in this chapter.